Сравнительный анализ оптимизаторов AdamW, Muon и гибридного подхода для параметрически-эффективного дообучения больших языковых моделей

Влад

23 марта 2026 г.

Аннотация

В данном отчёте представлено детальное исследование трёх стратегий оптимизации градиентного спуска для параметрически-эффективного дообучения (LoRA) больших языковых моделей. Рассматриваются: классический адаптивный оптимизатор AdamW, современный оптимизатор Muon на основе матричной ортогонализации, и предложенный гибридный подход, комбинирующий преимущества обоих методов. Для каждого оптимизатора приведены математические формулы, алгоритмы в псевдокоде и анализ вычислительной сложности. Эксперименты проведены на модели Qwen2.5-0.5B с использованием датасета OpenWebText и оценкой на задаче логического вывода PIQA. Результаты показывают, что AdamW обеспечивает наилучшую скорость обучения (0.682 шага/с), Muon экономит $\sim$2% видеопамяти, а гибридный подход демонстрирует сбалансированные характеристики. Все методы достигают сопоставимого качества на тестовой выборке ($\sim$70% accuracy), что указывает на достаточность данных методов для задач малого масштаба.

Содержание

1 Введение **3**

2 Теоретические основы **4**
2.1 Постановка задачи обучения 4
2.2 Проблемы стандартного стохастического градиентного спуска 4

3 Описание оптимизаторов **5**
3.1 AdamW: адаптивный момент с коррекцией веса 5
3.1.1 Идея метода 5
3.1.2 Математическая формулировка 5
3.1.3 Псевдокод 6
3.1.4 Вычислительная сложность и память 6
3.2 Muon: оптимизация через матричную ортогонализацию 6
3.2.1 Идея метода 6
3.2.2 Математическая формулировка 7
3.2.3 Псевдокод 7
3.2.4 Вычислительная сложность и память 8
3.3 Hybrid: комбинированный подход 8
3.3.1 Мотивация 8
3.3.2 Стратегия разделения параметров 8
3.3.3 Псевдокод гибридного оптимизатора 9
3.3.4 Преимущества и ограничения 9

4 Экспериментальная установка **10**
4.1 Аппаратное и программное обеспечение 10
4.2 Модель и данные 11
4.3 Метрики и протокол оценки 11

5 Результаты **12**
5.1 Сводная таблица метрик 12
5.2 Анализ производительности 12

6 Заключение **13**

1 Введение

Современные большие языковые модели (LLM) содержат сотни миллионов и миллиарды параметров, что делает их полное дообучение вычислительно дорогостоящим. Методы параметрически-эффективного дообучения, такие как LoRA (Low-Rank Adaptation) [2], позволяют адаптировать предобученные модели к новым задачам, обновляя лишь малую долю параметров (обычно <5%).

Ключевым компонентом процесса обучения является выбор оптимизатора — алгоритма, определяющего направление и величину обновления весов модели на основе градиентов функции потерь. В последние годы появились новые методы оптимизации, претендующие на улучшение сходимости и эффективности использования ресурсов по сравнению с классическими подходами.

Цель работы: провести сравнительный анализ трёх оптимизаторов для LoRA-дообучения:

1. **AdamW** [1] — стандартный адаптивный метод с коррекцией веса;
2. **Muon** [3] — оптимизатор на основе матричной ортогонализации;
3. **Hybrid** — авторский гибридный подход, комбинирующий Muon и AdamW.

Критерии сравнения: время обучения, пиковое потребление GPU-памяти, финальное значение функции потерь, точность на тестовой задаче (PIQA), скорость сходимости.

2 Теоретические основы

2.1 Постановка задачи обучения

Пусть дана модель с параметрами $\theta \in \mathbb{R}^d$ и функция потерь $\mathcal{L}(\theta; \mathcal{D})$, вычисляемая на обучающем датасете $\mathcal{D}$. Цель обучения — найти параметры θ^*, минимизирующие эмпирический риск:

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(f_\theta(x), y)], \tag{1}$$

где ℓ — функция потерь для отдельного примера, f_θ — предсказание модели.

В контексте дообучения с помощью LoRA исходные веса W_0 замораживаются, а к ним добавляется низкоранговое приращение:

$$W = W_0 + \Delta W = W_0 + BA, \quad A \in \mathbb{R}^{r\times k},\ B \in \mathbb{R}^{n\times r},\ r \ll \min(n, k). \tag{2}$$

Обучению подлежат только матрицы A и B, что существенно снижает количество обучаемых параметров.

2.2 Проблемы стандартного стохастического градиентного спуска

Классический SGD обновляет параметры по правилу:

$$\theta_{t+1} = \theta_t - \eta \cdot g_t, \quad g_t = \nabla_\theta \mathcal{L}(\theta_t), \tag{3}$$

где η — скорость обучения.

Основные ограничения SGD:

- **Чувствительность к масштабу градиентов**: параметры с разными масштабами требуют разных шагов обучения;
- **Шум мини-батчей**: стохастичность градиентов может приводить к колебаниям траектории;
- **Отсутствие адаптации**: единая скорость обучения для всех параметров неэффективна для разреженных или редко обновляемых весов.

Эти проблемы мотивируют использование адаптивных методов оптимизации.

3 Описание оптимизаторов

3.1 AdamW: адаптивный момент с коррекцией веса

3.1.1 Идея метода

AdamW [1] сочетает преимущества двух подходов:

- **Momentum**: накопление экспоненциально затухающего среднего градиентов для сглаживания обновлений;
- **Adaptive scaling**: нормировка градиентов на оценку их дисперсии для автоматической адаптации шага по каждому параметру;
- **Weight decay decoupling**: применение регуляризации отдельно от градиентного шага, что улучшает обобщение.

3.1.2 Математическая формулировка

На шаге t для каждого параметра θ_i вычисляются:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(первый момент)} \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(второй момент)} \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \text{(коррекция смещения)} \tag{6}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \text{(коррекция смещения)} \tag{7}$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right) \qquad \text{(обновление с weight decay)} \tag{8}$$

где:

- g_t — градиент на шаге t;
- $\beta_1, \beta_2 \in [0, 1)$ — коэффициенты затухания моментов (обычно 0.9 и 0.999);
- $\epsilon > 0$ — малая константа для численной стабильности (10^{-8});
- λ — коэффициент weight decay;
- η — базовая скорость обучения.

3.1.3 Псевдокод

Algorithm 1 AdamW (упрощённая версия)

Require: Параметры θ, градиенты g, гиперпараметры $\eta, \beta_1, \beta_2, \epsilon, \lambda$

Инициализировать $m \leftarrow 0$, $v \leftarrow 0$, $t \leftarrow 0$
while не достигнут критерий остановки **do**
  $t \leftarrow t + 1$
  $g \leftarrow \nabla_\theta \mathcal{L}(\theta)$ ▷ Вычисление градиента
  $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$
  $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$
  $\hat{m} \leftarrow m/(1 - \beta_1^t)$
  $\hat{v} \leftarrow v/(1 - \beta_2^t)$
  $\theta \leftarrow \theta - \eta \cdot \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda \cdot \theta \right)$
end while
return θ

3.1.4 Вычислительная сложность и память

- **Память**: $2d$ дополнительных скаляров на параметр (m и v);
- **Вычисления**: $O(d)$ операций на шаг (элементарные покомпонентные операции);
- **Параллелизация**: тривиальная, все операции покомпонентные.

3.2 Muon: оптимизация через матричную ортогонализацию

3.2.1 Идея метода

Muon (Matrix Orthogonalization for Updating Networks) [3] основан на гипотезе, что направление градиента можно улучшить путём ортогонализации матрицы обновлений. Метод применяет итерации Ньютона-Шульца для приближения полярного разложения градиентной матрицы, что теоретически ускоряет сходимость в задачах с коррелированными градиентами.

Ключевые особенности реализации в данном проекте:

- Применяется только к **двумерным тензорам** (весовым матрицам), исключая эмбеддинги и `lm_head`;
- Использует **один буфер момента** вместо двух, экономя память;
- Ортогонализация выполняется через **квинтичные итерации Ньютона-Шульца** с коэффициентами $(a, b, c) = (3.4445, -4.7750, 2.0315)$;
- Автоматическая корректировка learning rate в зависимости от размера матрицы: $\eta_{\text{adj}} = \eta \cdot 0.2 \cdot \sqrt{\max(n, m)}$.

3.2.2 Математическая формулировка

Пусть $G \in \mathbb{R}^{n \times m}$ — матрица градиентов для некоторого весового слоя.

Шаг 1: Нормировка и подготовка

$$X_0 = \frac{G}{\|G\|_F + \epsilon}, \quad \text{если } n > m, \text{ то } X_0 \leftarrow X_0^\top \tag{9}$$

Шаг 2: Квинтичные итерации Ньютона-Шульца

Для $k = 0, 1, \ldots, K-1$:

$$A_k = X_k X_k^\top \tag{10}$$

$$B_k = b \cdot A_k + c \cdot A_k^2 \tag{11}$$

$$X_{k+1} = a \cdot X_k + B_k X_k \tag{12}$$

где коэффициенты $(a, b, c) = (3.4445, -4.7750, 2.0315)$ подобраны для максимизации скорости сходимости.

Шаг 3: Обновление с моментом и корректировкой шага

$$M_t = \beta M_{t-1} + (1-\beta) \cdot X_K \tag{13}$$

$$\eta_{\text{adj}} = \eta \cdot 0.2 \cdot \sqrt{\max(n, m)} \tag{14}$$

$$\theta_{t+1} = \theta_t - \eta_{\text{adj}} \cdot M_t - \eta \cdot \lambda \cdot \theta_t \tag{15}$$

3.2.3 Псевдокод

Algorithm 2 Muon: обновление одного весового матричного параметра

Require: Матрица весов W, градиент G, гиперпараметры $\eta, \beta, \lambda, K, (a, b, c)$

if G не является 2D-тензором **then**
  return AdamW-обновление для G ▷ Fallback для векторов
end if
▷ Шаг 1: нормировка
$X \leftarrow G/(\|G\|_F + \epsilon)$
if $\text{rows}(G) > \text{cols}(G)$ **then**
  $X \leftarrow X^\top$
end if
▷ Шаг 2: квинтичные итерации Ньютона-Шульца
for $k = 1$ **to** K **do**
  $A \leftarrow X \cdot X^\top$
  $B \leftarrow b \cdot A + c \cdot (A \cdot A)$
  $X \leftarrow a \cdot X + B \cdot X$
end for
if $\text{rows}(G) > \text{cols}(G)$ **then**
  $X \leftarrow X^\top$
end if
▷ Шаг 3: момент и обновление
$M \leftarrow \beta \cdot M + (1-\beta) \cdot X$
$\eta_{\text{adj}} \leftarrow \eta \cdot 0.2 \cdot \sqrt{\max(\text{shape}(W))}$
$W \leftarrow W - \eta_{\text{adj}} \cdot M - \eta \cdot \lambda \cdot W$
return W

3.2.4 Вычислительная сложность и память

- **Память**: d скаляров на параметр (один момент M) + временные буферы $O(nm)$ для итераций;
- **Вычисления**: $O(K \cdot nm \cdot \min(n, m))$ на шаг из-за матричных умножений в итерациях Ньютона-Шульца;
- **Параллелизация**: матричные операции хорошо параллелятся на GPU, но итеративная природа ограничивает конвейеризацию.

Практические замечания:

- Число итераций K по умолчанию равно 5; больше итераций дают лучшее приближение, но увеличивают время;
- Для стабильности используется нормировка спектральной нормы и $\epsilon = 10^{-7}$;
- Метод не применяется к 1D-параметрам (bias, layer norm), где ортогонализация не имеет смысла.

3.3 Hybrid: комбинированный подход

3.3.1 Мотивация

Ни AdamW, ни Muon не являются универсально оптимальными:

- AdamW быстр и стабилен, но может медленнее сходиться в задачах с сильными корреляциями градиентов;
- Muon теоретически ускоряет сходимость, но требует больше вычислений и может быть нестабилен на ранних этапах обучения.

Идея гибридного подхода: использовать сильные стороны каждого метода, назначая их разным группам параметров.

3.3.2 Стратегия разделения параметров

Параметры модели разделяются на две группы согласно алгоритму из `src/optimizers/optimizer.`

1. **Фильтрация 2D-параметров**: из всех параметров отбираются только двумерные тензоры, исключая те, что содержат в имени `embed_tokens` или `lm_head`:

```
all_2d_params = [
    p for p in params
    if p.ndim >= 2
    and "embed_tokens" not in str(p)
    and "lm_head" not in str(p)
]

```

2. **Разделение пополам**: первые 50% отобранных 2D-параметров обучаются через Muon, остальные 50% + все 1D-параметры — через AdamW:

```
split_idx = len(all_2d_params) // 2
muon_params = all_2d_params[:split_idx]            #
                          Muon
adamw_params = all_2d_params[split_idx:] + other_params  #
                                  +                        AdamW

```

3.3.3 Псевдокод гибридного оптимизатора

Algorithm 3 Hybrid: комбинированное обновление всех параметров

Require: Параметры $\Theta = \{\theta_1, \ldots, \theta_N\}$, градиенты $\{g_1, \ldots, g_N\}$

$\triangleright$ Фильтрация 2D-параметров

$all_2d \leftarrow \{p \in \Theta \mid p.\text{ndim} \geq 2 \wedge \text{"embed_tokens} \notin p \wedge \text{"lm_head} \notin p\}$

$other \leftarrow \Theta \setminus all_2d$

$\triangleright$ Разделение пополам

$split \leftarrow \lfloor |all_2d|/2 \rfloor$

$muon_params \leftarrow all_2d[0 : split]$

$adamw_params \leftarrow all_2d[split :] \cup other$

$\triangleright$ Обновление через единый интерфейс Muon

return MUON.STEP($muon_params$, $adamw_params$, g)

3.3.4 Преимущества и ограничения

Преимущества:

- **Баланс скорости и качества**: критически важные слои получают более мощную оптимизацию (Muon), остальные — быструю (AdamW);
- **Гибкость**: стратегию разделения можно адаптировать под архитектуру;
- **Обратная совместимость**: при $split = 0$ метод вырождается в чистый AdamW.

Ограничения:

- **Эвристика разделения**: выбор первых 50% матриц может быть неоптимальным для всех архитектур;
- **Дополнительная сложность**: требуется отслеживание типов и порядков параметров;
- **Отсутствие теоретических гарантий**: комбинация двух оптимизаторов не имеет формального обоснования сходимости.

4 Экспериментальная установка

4.1 Аппаратное и программное обеспечение

Таблица 1: Аппаратная конфигурация

Компонент	**Характеристики**
Процессор	Intel Core i5-11400F (6 ядер / 12 потоков)
Оперативная память	16 ГБ DDR4
Видеокарта	NVIDIA GeForce RTX 5060 Ti, 16 ГБ VRAM
Операционная система	Windows 10 Pro

Таблица 2: Зависимости программного обеспечения (из `requirements.txt`)

Пакет	**Версия**
torch	2.10.0+cu128
datasets	4.8.2
hydra-core	1.3.2
lm_eval	0.4.11
matplotlib	3.10.8
mypy	1.19.1
peft	0.18.1
psutil	7.2.2
pylint	4.0.5
tokenizers	0.22.2
transformers	5.3.0
trl	0.29.0

Примечание: пакет `torch` установлен из дополнительного индекса `https://download.pytorch.org/whl/cu128` для поддержки CUDA 12.8.

4.2 Модель и данные

Таблица 3: Параметры экспериментальной установки

Компонент	**Значение**
Модель	Qwen/Qwen2.5-0.5B (502M параметров)
Адаптация	LoRA: $r = 16$, $\alpha = 32$, dropout=0.05
Целевые модули	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Обучаемые параметры	8.8M (1.75% от общего числа)
Тренировочный датасет	Elriggs/openwebtext-100k (1% = 1000 сэмплов)
Тестовый датасет	PIQA (1838 примеров логического вывода)
Точность вычислений	bfloat16 (нативная поддержка на RTX 5060 Ti)
Размер батча	4 на устройство, 4 шага накопления градиентов
Эпохи	5
Learning rate	5×10^{-5}, cosine schedule с warmup 3%
Weight decay	0.01
Оптимизаторы	AdamW / Muon / Hybrid

4.3 Метрики и протокол оценки

Для каждого эксперимента фиксируются:

1. **Время обучения**: от начала первого шага до завершения последней эпохи;
2. **Пиковая память GPU**: максимальное значение;
3. **Финальный loss**: среднее значение функции потерь на последнем шаге;
4. **PIQA Accuracy**: доля правильных ответов на тестовом датасете после дообучения;
5. **Шагов в секунду**: усреднённая скорость обработки мини-батчей.

Все эксперименты запускаются с фиксированным random seed (42) для воспроизводимости.

5 Результаты

5.1 Сводная таблица метрик

Таблица 4: Сравнение оптимизаторов по ключевым метрикам

Метрика	**AdamW**	**Muon**	**Hybrid**
Время обучения	**7м 41с**	9м 22с	8м 23с
Пиковая память, МБ	3912	**3846**	3878
Финальный loss	**2.863**	2.903	2.888
PIQA Accuracy, %	**70.29**	70.13	69.86
Шагов в секунду	**0.682**	0.560	0.626
Trainable params	8.8M (1.75% от общего числа)		

5.2 Анализ производительности

Скорость обучения. AdamW демонстрирует наилучшую производительность: 0.682 шага/с против 0.560 у Muon (-18%). Это объясняется отсутствием итераций Ньютона-Шульца, требующих дополнительных матричных умножений. Гибридный подход занимает промежуточное положение (0.626 шага/с), так как лишь половина параметров обрабатывается более тяжёлым Muon.

Потребление памяти. Muon экономит $\sim$66 МБ ($\sim$1.7%) видеопамяти по сравнению с AdamW благодаря использованию одного буфера момента вместо двух. Гибридный метод потребляет промежуточное количество памяти, пропорциональное доле параметров, обрабатываемых каждым оптимизатором.

Качество обучения. Все три метода достигают сопоставимого финального loss ($\sim$2.86–2.90) и точности на PIQA ($\sim$70%). Небольшие различия (менее 0.5% accuracy) статистически незначимы при данном объёме данных (1000 тренировочных сэмплов). Это указывает на то, что для задач малого масштаба выбор оптимизатора влияет в первую очередь на эффективность, а не на итоговое качество.

Динамика сходимости. Анализ логов показывает, что все оптимизаторы демонстрируют монотонное снижение loss без признаков расходимости. На ранних этапах (первые 2 эпохи) Muon иногда показывает более резкие колебания, что может быть связано с итеративной природой ортогонализации. К концу обучения траектории всех методов стабилизируются.

6 Заключение

В данной работе проведён детальный сравнительный анализ трёх оптимизаторов для LoRA-дообучения больших языковых моделей:

1. **AdamW** остаётся надёжным выбором для большинства практических задач: он быстр, стабилен и хорошо изучен. Его покомпонентная адаптивность эффективно справляется с разномасштабными градиентами, а коррекция weight decay улучшает обобщение.

2. **Muon** предлагает инновационный подход через матричную ортогонализацию, теоретически ускоряющий сходимость в задачах с коррелированными градиентами. На практике он экономит память и показывает сопоставимое качество, но требует больше вычислений.

3. **Hybrid** демонстрирует, что комбинация методов может давать сбалансированные результаты. Предложенная стратегия разделения параметров (первые 50% 2D-матриц $\rightarrow$ Muon, остальные $\rightarrow$ AdamW) является простой, но эффективной эвристикой.

Ключевой вывод: для задач малого и среднего масштаба (как в данном эксперименте) различия в итоговом качестве между оптимизаторами минимальны. Выбор следует делать исходя из ограничений по времени и памяти: AdamW для скорости, Muon для экономии ресурсов, Hybrid для баланса.

Практическая рекомендация: начинайте с AdamW как с базового варианта. При наличии избыточного времени или требований к минимизации памяти экспериментируйте с Muon или гибридными схемами, обязательно проводя валидацию на отложенной выборке.

Список литературы

Список литературы

[1] Loshchilov I., Hutter F. *Decoupled Weight Decay Regularization.* International Conference on Learning Representations (ICLR), 2019.

[2] Hu E. J. et al. *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv preprint arXiv:2106.09685, 2021.

[3] MoonshotAI. *Muon: Matrix Orthogonalization for Efficient LLM Training.* GitHub repository, 2024. `https://github.com/MoonshotAI/Moonlight`

[4] Bisk Y. et al. *PIQA: Reasoning about Physical Commonsense in Natural Language.* Proceedings of the AAAI Conference on Artificial Intelligence, 2020.

[5] Malladi S. et al. *MeZO: Memory-Efficient Zeroth-Order Optimizer for LLM Fine-tuning.* arXiv preprint arXiv:2305.17333, 2023.

[6] Schulz G. *Iterative Berechnung der reziproken Matrix.* Zeitschrift für Angewandte Mathematik und Mechanik, 1933.

Приложение: Исходные логи экспериментов

Полные логи обучения и оценки доступны в репозитории:

- `outputs/checkpoints/metrics_adamw.json`
- `outputs/checkpoints/metrics_muon.json`
- `outputs/checkpoints/metrics_hybrid.json`
- `logs/train_*.log`

Для воспроизведения результатов выполните:

```
#
git clone https://github.com/Not-broken-today/llm-training-acceleration
cd llm-training-acceleration

#
pip install -r requirements.txt --extra-index-url https://download.
    pytorch.org/whl/cu128

#                                        (                    AdamW)
bash run_file/AdamW_run.sh

#
bash pylint_test_code.sh
mypy src/
```